

Valkey on Kubernetes

High-Performance, Low-Latency Deployments for Scalable Clusters

马丽丽

亚马逊云科技

数据库解决方案架构师

肖萍

亚马逊云科技

计算服务解决方案架构师



What We'll Cover Today

01

What is Valkey?

Origins, licensing, Linux Foundation governance, and the Redis fork story

02

Cluster Architecture

Standalone vs. Cluster mode, sharding, replication, and Sentinel topologies

03

Kubernetes Deployment Patterns

StatefulSets, Operators, PVCs, and pod scheduling strategies

04

Performance Tuning

CPU pinning, memory limits, kernel parameters, and benchmarking

05

Observability & Operations

Prometheus metrics, alerts, backup strategies, and Day 2 runbooks





Valkey

Open source, high-performance in-memory key/value data store



What is Valkey ?

🔑 BSD Licensed

Fully open-source with no cloud provider restrictions or source-available caveats

🏛️ Linux Foundation

Neutral governance with major backing from AWS, Google, Oracle, Ericsson, Tencent and Alibaba

⚡ Redis-Compatible

Drop-in protocol and API compatibility existing clients work without modification

🚀 Active Roadmap

Multi-threaded I/O improvements, new data structures, and performance milestones



Why Valkey on Kubernetes?

The Platform Case

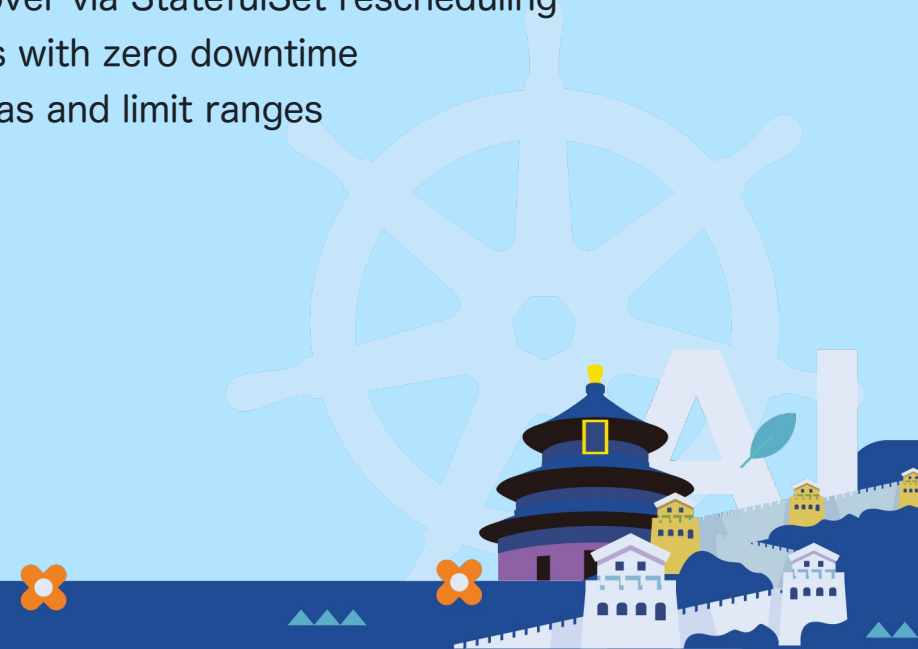
Kubernetes abstracts infrastructure, enabling consistent Valkey deployments across public clouds, and on-prem bare metal with a single set of manifests. GitOps workflows apply equally to caching infrastructure as they do to application services.

- Unified control plane for compute and data
- Namespace isolation for multi-tenancy
- Pod disruption budgets for safe upgrades

The Operational Case

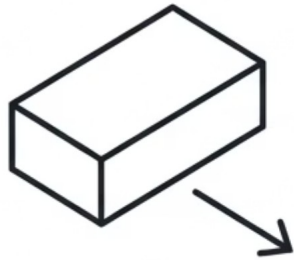
SRE teams benefit from Kubernetes-native liveness and readiness probes, automatic pod rescheduling on node failure, and horizontal pod autoscaling for read replicas. Combined with Valkey's own cluster-mode replication, you get layered resilience at both the application and infrastructure level.

- Automatic failover via StatefulSet rescheduling
- Rolling restarts with zero downtime
- Resource quotas and limit ranges



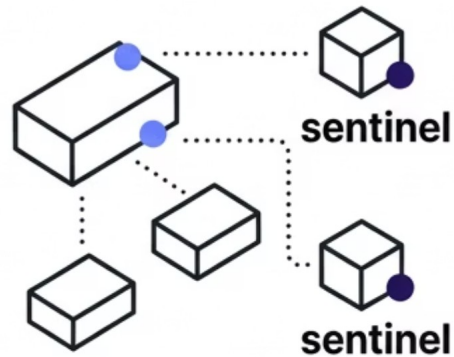
Cluster Topology Options

Standalone



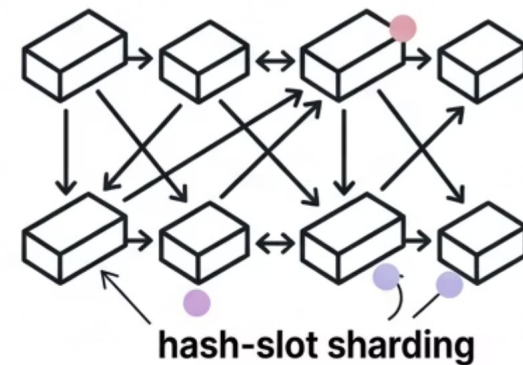
- Single Node
- Simple Ops
 - No HA
- Dev/Test Best

Sentinel



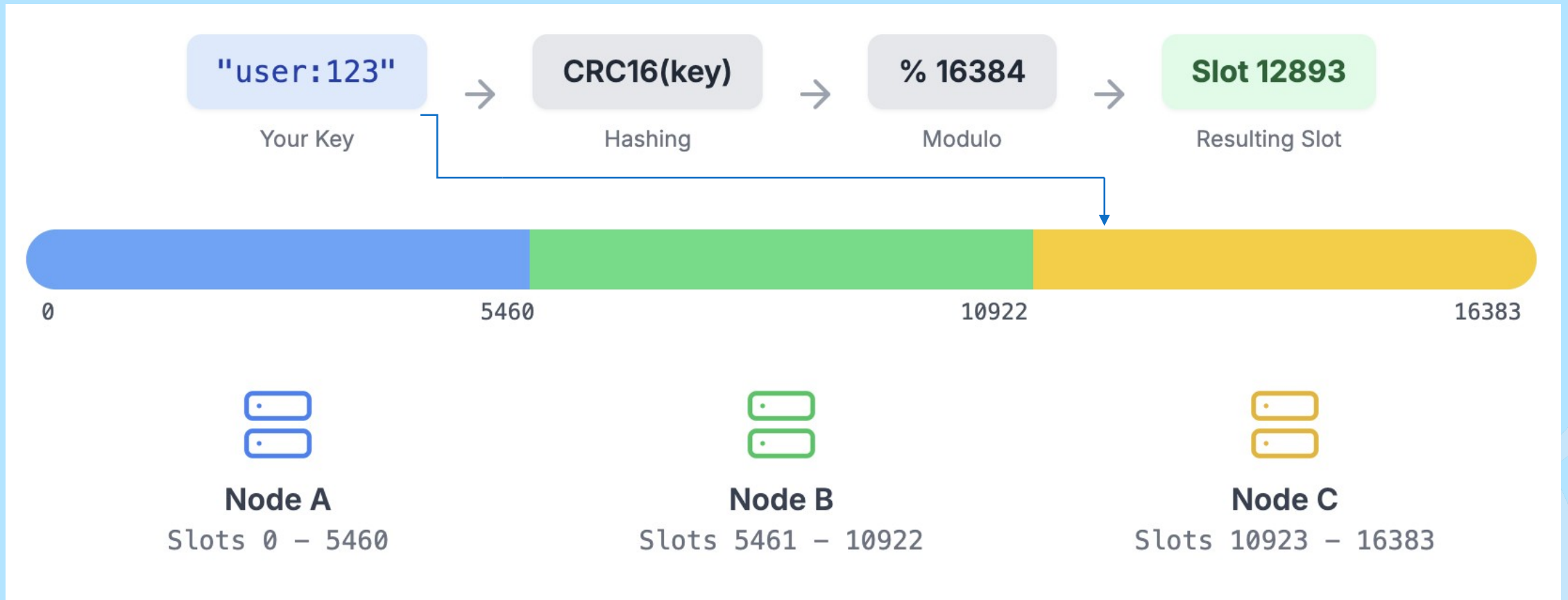
- Automatic Failover
- Manual Sharding
- Moderate Scale

Cluster Mode



- Primary-Replica Pairs
- Automatic Rebalancing
- High Throughput
- Horizontal Scale

Valkey Cluster How do Cluster Slots Work



StatefulSets & Operators

Deployment Patterns

StatefulSet Core Config

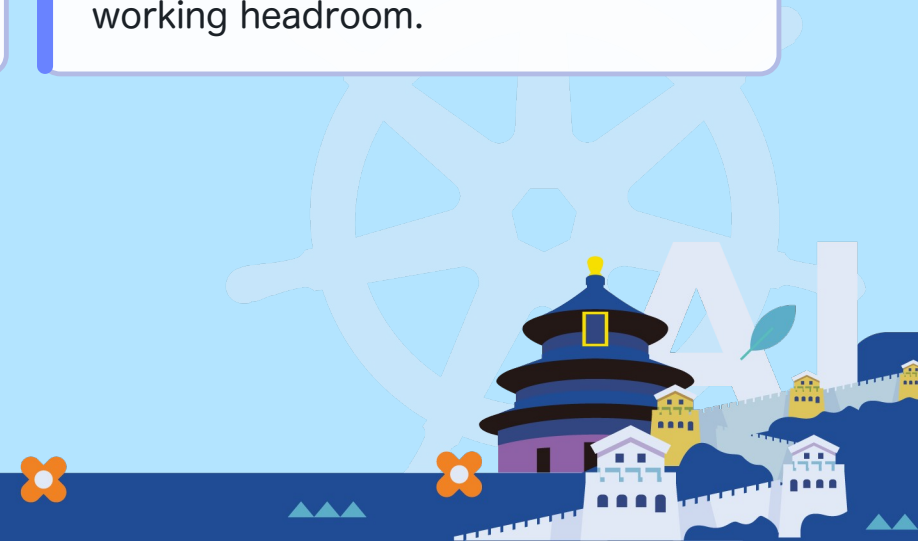
Set `podManagementPolicy`: `Parallel` for faster cluster bootstrap. Use headless Services (`clusterIP`: `None`) so pods can address each other directly by DNS, which Valkey Cluster requires for gossip and redirects.

Valkey Operator

The community Valkey Operator (work in progress) manages cluster initialization, failover promotion, and shard rebalancing via CRDs. Prefer operators for Day 2 operations — manual cluster management at scale is error-prone.

Persistent Volume Claims

Use `volumeClaimTemplates` with a fast StorageClass (NVMe-backed `io2/pd-ssd`). Set `reclaimPolicy`: `Retain` to prevent data loss on accidental StatefulSet deletion. Size PVCs for AOF + RDB snapshots plus working headroom.



Practical Deployment Strategies



Direct Kubernetes YAML

For ultimate control, define `StatefulSets`, `Services`, and `ConfigMaps` directly. This approach offers maximum flexibility for highly customized Valkey deployments but demands deep Kubernetes expertise and manual management for updates.



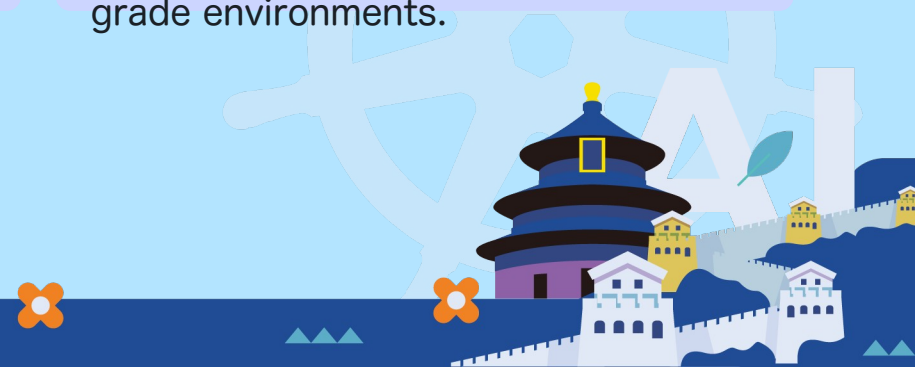
Helm Charts

Standardize Valkey deployments with parameterized templates. The **Valkey Helm chart** simplifies installation, configuration, and upgrades, incorporating community-vetted best practices for various topologies. Highly recommended for repeatable, version-controlled deployments.



Valkey Operators

Leverage the emerging Valkey Operator (currently under active development) to automate the entire lifecycle of Valkey clusters. This declarative approach handles provisioning, scaling, failover, and upgrades, reducing operational overhead for complex, production-grade environments.



Pod Scheduling & Resource Strategy

Anti-Affinity Rules

Always configure `podAntiAffinity` to spread primary shards across failure domains. Use `requiredDuringSchedulingIgnoredDuringExecution` to hard-enforce shard separation across availability zones or physical nodes.

- ❏ A single AZ failure should never take out more than one primary shard.

Resource Requests & Limits

Set CPU requests equal to limits (Guaranteed QoS class) to prevent throttling under load. Memory limits must account for maxmemory plus OS page cache overhead — typically add 20–30% headroom above your configured maxmemory value.

- CPU: requests = limits (Guaranteed QoS)
- Memory: maxmemory + 25% overhead
- Use `cpu-manager-policy: static` for exclusive cores

*QoS (Quality of Service)



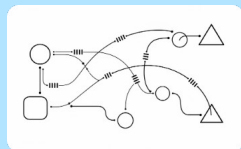
Squeezing Every Microsecond

Performance Tuning



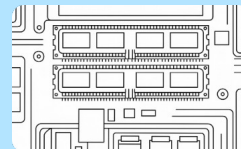
Kernel & OS Tuning

Apply `net.core.somaxconn=65535`, `vm.overcommit_memory=1`, and disable Transparent Huge Pages via a privileged init container. Use a `RuntimeClass` with `runc` for lowest overhead.



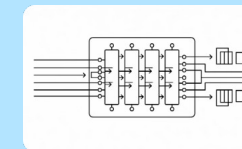
Network Latency

Enable host networking (`hostNetwork: true`) for latency-critical workloads to bypass kube-proxy overhead. Alternatively, use `endpointslice` and eBPF-based CNIs (Cilium) to minimize forwarding hops.

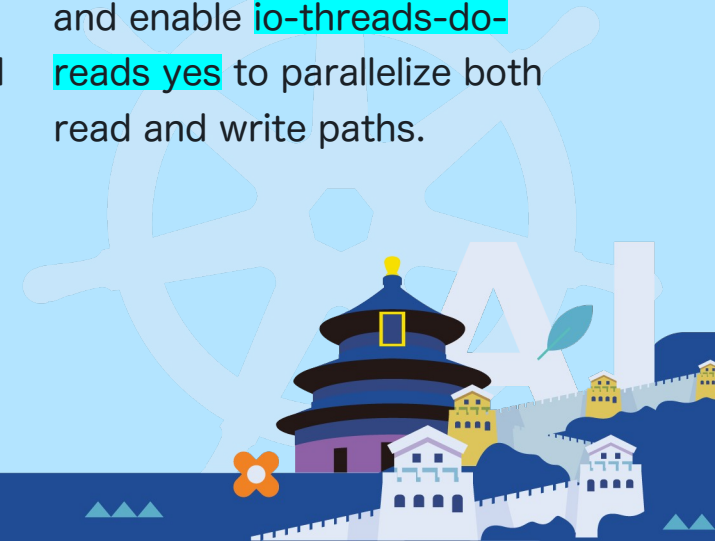


Memory ManagementMulti-Threaded I/O

Set `maxmemory-policy` based on workload: `allkeys-lru` for pure cache, `volatile-lru` for mixed storage/cache. Enable `lazyfree-lazy-eviction yes` to avoid blocking the main thread during eviction storms.



Valkey 8.x introduces improved multi-threaded I/O. Set `io-threads` to the number of physical cores minus one, and enable `io-threads-do-reads yes` to parallelize both read and write paths.



Benchmarking Your Cluster

Running valkey-benchmark

```
valkey-benchmark \  
  -h valkey-cluster.default.svc \  
  -p 6379 \  
  -c 100 \  
  -n 1000000 \  
  -t get,set \  
  --cluster \  
  -P 16 \  
  -q
```

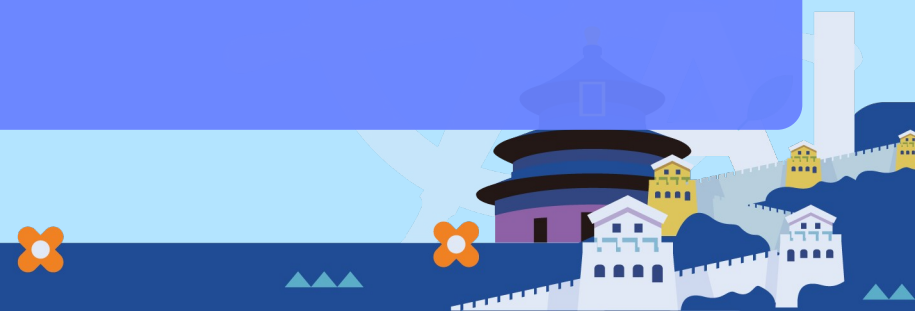
Key flags: `-c` concurrent connections, `-P` pipeline depth, `--cluster` for cluster-mode topology awareness. Run from within the cluster to eliminate external network variance.

Baseline Targets

On a well-tuned 3-shard cluster with NVMe storage and Guaranteed QoS pods, expect:

- **GET/SET throughput:** 500K–1M ops/sec
- **p50 latency:** < 200µs
- **p99 latency:** < 1ms
- **Memory overhead:** ~3–5% per key metadata

Deviations from these baselines indicate misconfigured kernel params, CPU throttling, or network policy overhead.



Metrics, Alerts & Dashboards

Observability



Key Metrics to Track

`instantaneous_ops_per_sec`,
`used_memory_rss`,
`keyspace_hits/misses`,
`connected_clients`,
`blocked_clients`, replication lag



Critical Alerts

Alert on: memory usage > 80% of `maxmemory`, replication lag > 100ms, evicted keys spike, cluster state != ok, and AOF rewrite duration outliers

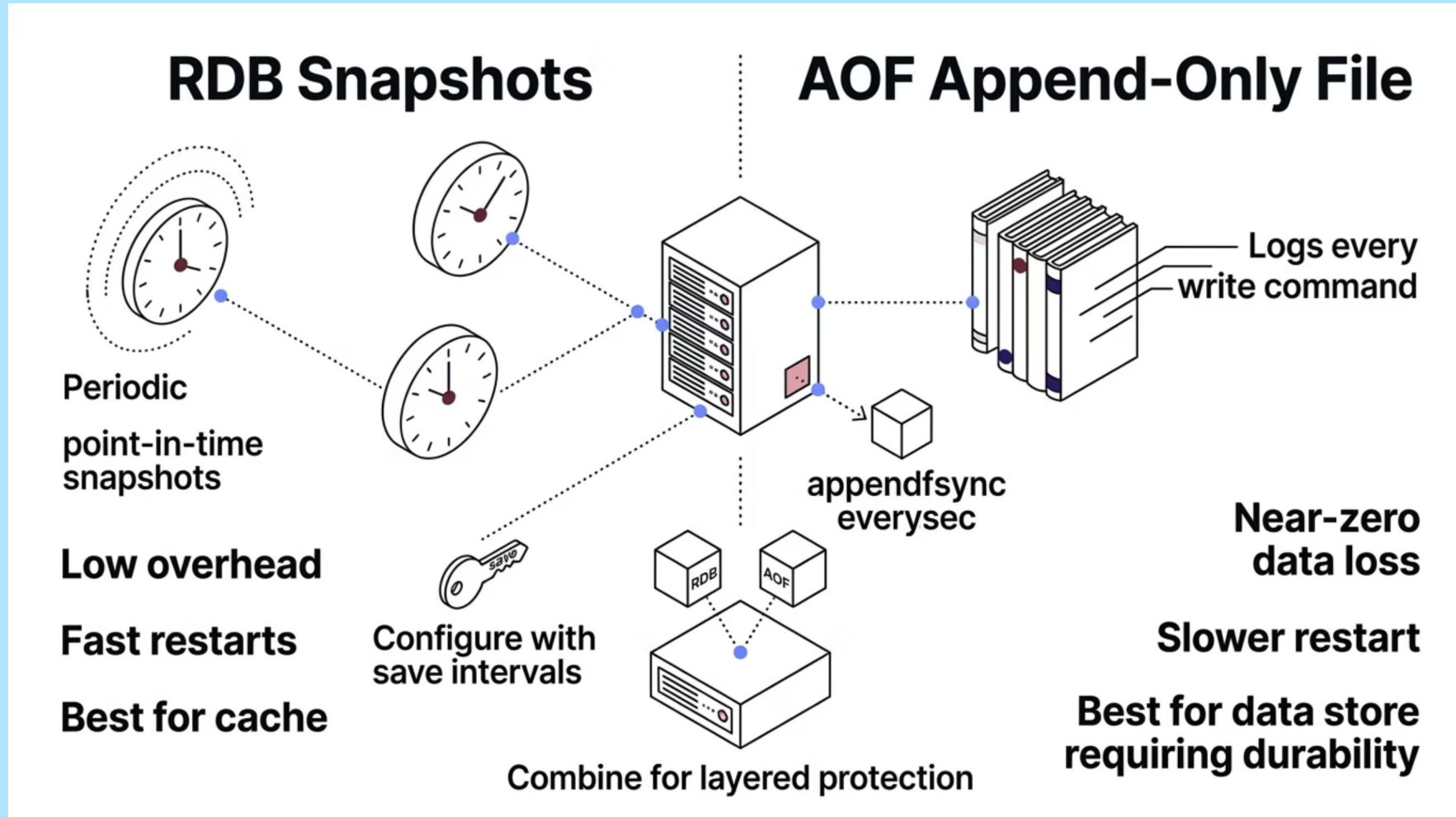


Grafana Dashboards

Import community dashboard ID **763** (redis-exporter) as a starting point. Add Kubernetes node-level metrics to correlate pod CPU throttling with latency spikes



Persistence & Backup Strategy



For Kubernetes deployments, use [Velero](#) with **CSI volume snapshots** to back up PVCs on a schedule. Store RDB snapshots to object storage (S3/GCS) via a sidecar or CronJob for offsite recovery capability.

Failover & Replication

High Availability

Cluster Failover Timeline

Default failover completes in **~15 seconds**: 5s node timeout + election + promotion. For tighter SLAs, reduce cluster-node-timeout to 2000ms, but be cautious of false positives during node pressure events on Kubernetes.

- NODE_TIMEOUT: default 15s, tune to 2–5s
- Replica promotion is automatic in cluster mode
- Clients must handle MOVED/ASK redirects

PodDisruptionBudgets

Always define a PDB for your Valkey StatefulSet to prevent Kubernetes from evicting too many pods simultaneously during node drain operations or cluster upgrades. A minimum available of **N-1** primaries ensures the cluster remains writable throughout voluntary disruptions.

- Set **minAvailable** per shard group
- Coordinate with node drain automation
- Test with **kubectl drain --ignore-daemonsets**



Scaling: Horizontal & Vertical

1

Vertical Scaling

Resize CPU/memory requests on the StatefulSet. Requires a rolling restart, plan for failover capacity. Use VPA (Vertical Pod Autoscaler) in recommendation mode first before enabling auto-apply.

2

Add Replicas

Scale read throughput by increasing replica count per shard. New replicas join via full resync. HPA can scale replicas based on custom metrics like `keyspace_hits` rate from Prometheus Adapter.

3

Add Shards

Horizontal shard scaling requires hash slot rebalancing via `valkey-cli --cluster rebalance`. This is a live operation but incurs replication traffic, schedule during low-traffic windows. Operators can automate this lifecycle.

❏ **Important:** Clients using cluster-mode-aware drivers (Lettuce, StackExchange.Redis, go-redis) handle MOVED redirects automatically. Ensure all application teams are using cluster-compatible client configurations before scaling shards.



Security Hardening

ACL-Based Access Control

Use Valkey's built-in ACL system to define per-user command and key-pattern permissions. Store ACL configs in a Kubernetes Secret mounted as a volume. Rotate credentials via secret version bumps with a rolling restart strategy.

TLS Encryption

Enable TLS for both client-server and intra-cluster (bus) communication. Use cert-manager to provision and auto-rotate certificates. Note that TLS adds ~10–15% CPU overhead, account for this in resource requests.

NetworkPolicy Isolation

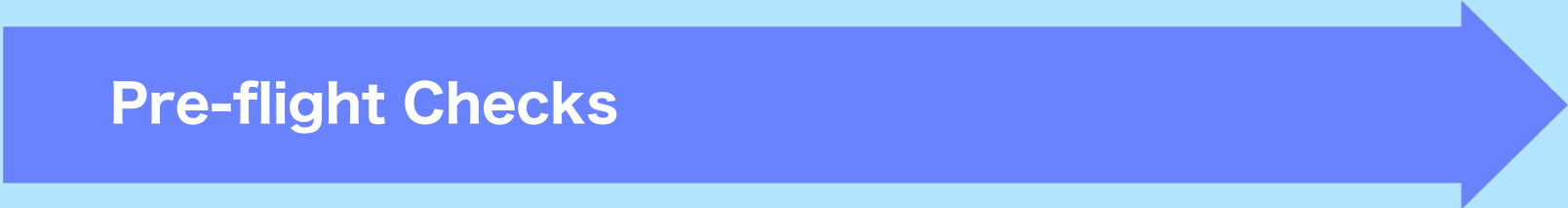
Apply Kubernetes NetworkPolicies to restrict ingress to Valkey pods, only allow traffic from known application namespaces and monitoring infrastructure. Deny all egress except to cluster peers and DNS.



Upgrade & Maintenance Runbook

Day 2 Operations

Pre-flight Checks



Rolling Replica Upgrade



Primary Failover



Validate & Monitor



Key Takeaways & Next Steps

✓ Start with Cluster Mode

Use 3 primaries + 1 replica each as your baseline production topology. Grow shards before vertical sizing.

✓ StatefulSets + Operator

Automate Day 2 operations with a Valkey Operator. Manual cluster management doesn't scale with your teams.

✓ Tune the Full Stack

Kernel params, QoS class, PDBs, and ACLs are all required. Partial configuration leaves performance on the table.

✓ Observe Everything

Deploy redis-exporter, set replication lag alerts, and build a Grafana dashboard before your first production incident.

[GITHUB.COM/
VALKEY-IO](https://github.com/valkey-io/valkey-io)

[VALKEY.IO](https://valkey.io)

[LINC FOUNDATION
PROJECT](https://linc.foundation)



Thanks

